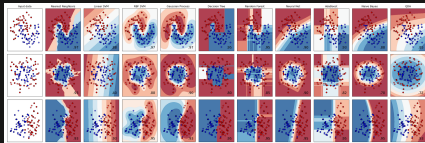


Árboles de Decisión y Ensamblados: Bagging, Random Forest & GBDT (XGBoost/LightGBM/CatBoost)

Aprendizaje Automatico



Marco Teran
EAFIT

Contenido

- 1 Objetivos de Aprendizaje
- 2 Decision Trees
 - Medidas de Impureza
 - Regularización en Decision Trees
- 3 Ensemble Learning
 - Voting Classifiers
- 4 Bagging
- 5 Random Forest
- 6 Boosting
- 7 Gradient Boosting
- 8 XGBoost
- 9 LightGBM
- 10 CatBoost
- 11 Comparación
- 12 Conclusiones

Decision Trees & Ensemble Learning

Objetivos de aprendizaje

- **Dominar fundamentos matemáticos** de árboles de decisión y métodos de ensamble
- **Comprender criterios de división:** Gini impurity vs Entropy, derivaciones completas
- **Implementar algoritmos desde cero:** CART, Bagging, Boosting
- **Aplicar técnicas avanzadas:** Random Forest, XGBoost, LightGBM, CatBoost
- **Optimizar para producción:** feature importance, hyperparameter tuning, early stopping
- **Diagnosticar problemas:** overfitting, bias-variance tradeoff, model selection

Filosofía central

Construir modelos robustos mediante la agregación inteligente de predictores débiles

Decision Trees: Fundamentos

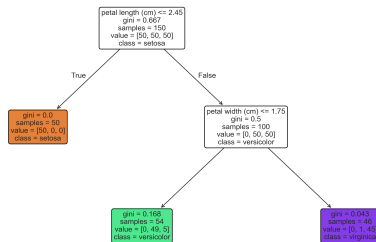
¿Qué es un Decision Tree?

Un árbol de decisión es un modelo de **aprendizaje supervisado** que realiza predicciones mediante una *estructura jerárquica de reglas de decisión*.

Componentes principales:

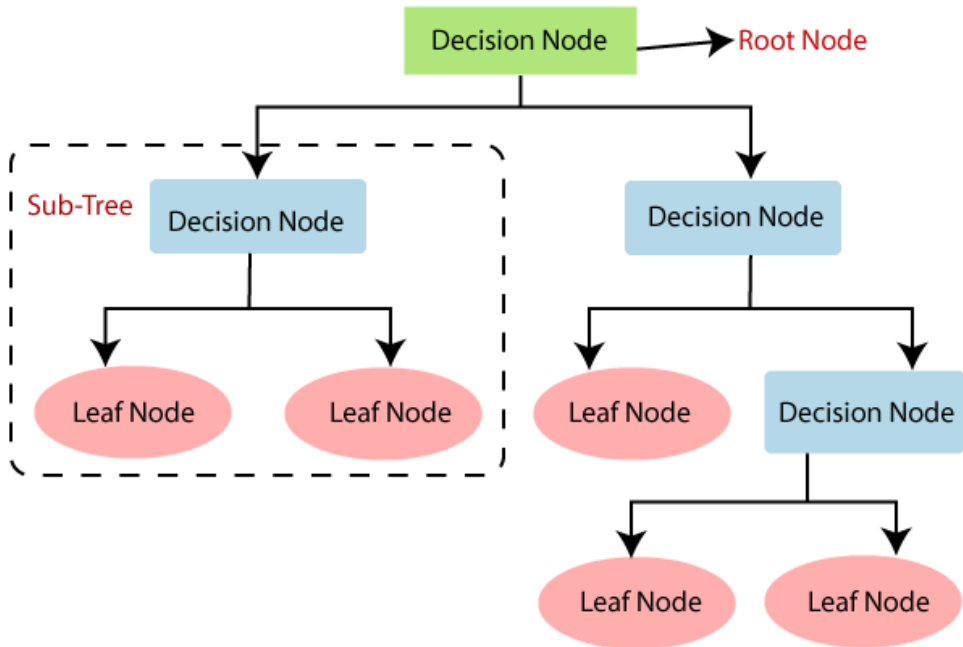
- **Nodo raíz:** primera decisión
- **Nodos internos:** reglas de división
- **Nodos hoja:** predicciones finales
- **Ramas:** caminos de decisión

Decision Tree on Iris Dataset (max_depth=2)



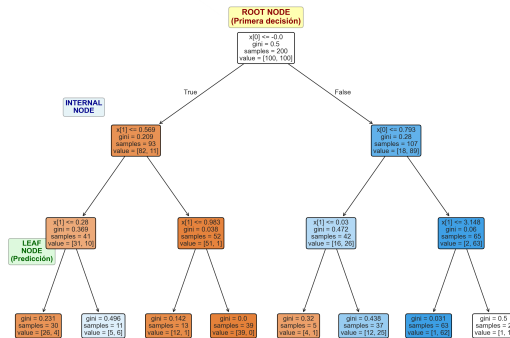
Ventajas clave

Interpretable, no requiere normalización, maneja features categóricas y numéricas



Anatomía de un árbol de decisión

Anatomía de un Árbol de Decisión



Proceso de predicción:

- 1 Comenzar en el nodo raíz
- 2 Evaluar condición del nodo actual
- 3 Seguir rama correspondiente (True/False)
- 4 Repetir hasta alcanzar un nodo hoja

CART: Algoritmo fundamental

CART (*Classification and Regression Trees*) construye árboles binarios mediante particiones recursivas que minimizan una función de costo.

CART: Algoritmo fundamental

Función de costo para clasificación:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

Donde:

- k : feature seleccionado para el split
- t_k : threshold (umbral) para dividir
- G : medida de impureza (Gini o Entropy)
- m : número total de muestras
- $m_{\text{left}}, m_{\text{right}}$: muestras en cada hijo

CART: Algoritmo paso a paso

Procedimiento recursivo:

- 1 **Inicio:** Conjunto completo de datos en raíz
- 2 **Para cada nodo:**
 - Evaluar todos los features
 - Para cada feature, probar todos los *thresholds* posibles
 - Seleccionar (k, t_k) que minimiza $J(k, t_k)$
 - Crear dos nodos hijos con las particiones
- 3 **Recursión:** Repetir para cada hijo
- 4 **Criterio de parada:**
 - Profundidad máxima alcanzada
 - Número mínimo de muestras no cumplido
 - No hay reducción significativa de impureza
 - Nodo puro (impureza = 0)

Propiedades de CART

Ventajas:

- Árboles binarios (fácil implementación)
- Maneja *features* numéricos y categóricos
- Solución determinística
- No requiere normalización
- Robusto a *outliers*

Desventajas:

- *Greedy algorithm* (no garantiza óptimo global)
- Sensible a rotaciones de datos
- Propenso a *overfitting*
- Inestable (pequeños cambios $\rightarrow$ árbol diferente)

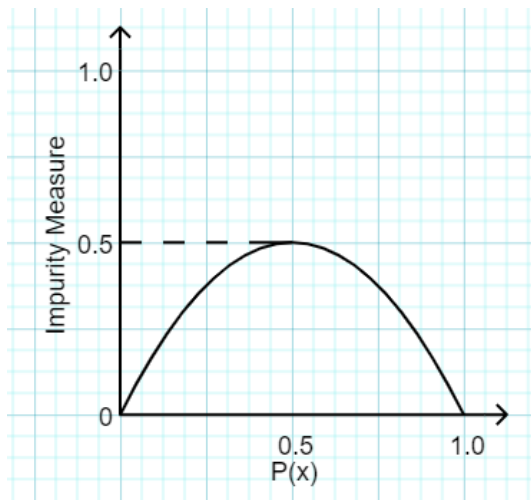
Complejidad computacional

Entrenamiento: $O(n \cdot m \log_2(m))$ donde n = features, m = muestras

Gini Impurity vs Entropy

Gini Impurity: Definición y derivación

La impureza de Gini mide la **probabilidad** de clasificar incorrectamente una muestra seleccionada aleatoriamente.



Gini Impurity: Definición y derivación

Fórmula:

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

Donde:

- $p_{i,k}$: proporción de muestras de clase k en el nodo i
- n : número total de clases
- Rango: $[0, 1 - \frac{1}{n}]$
- $G_i = 0$ indica pureza perfecta (todas las muestras de una clase)

Gini Impurity: Interpretación probabilística

Derivación intuitiva:

Si seleccionamos dos muestras al azar del nodo i :

- Probabilidad de que la primera sea clase k : $p_{i,k}$
- Probabilidad de que la segunda NO sea clase k : $(1 - p_{i,k})$
- Probabilidad de clasificación incorrecta para clase k : $p_{i,k}(1 - p_{i,k})$

Sumando sobre todas las clases:

$$G_i = \sum_{k=1}^n p_{i,k}(1 - p_{i,k}) = \sum_{k=1}^n p_{i,k} - \sum_{k=1}^n p_{i,k}^2 = 1 - \sum_{k=1}^n p_{i,k}^2$$

Casos extremos

Máxima impureza (distribución uniforme): $G = 1 - \frac{1}{n}$

Mínima impureza (nodo puro): $G = 0$

Gini Impurity: Ejemplo numérico

Escenario: Nodo con 54 muestras

- Clase 0 (setosa): 0 muestras $\rightarrow p_{i,0} = 0/54 = 0$
- Clase 1 (versicolor): 49 muestras $\rightarrow p_{i,1} = 49/54 \approx 0.907$
- Clase 2 (virginica): 5 muestras $\rightarrow p_{i,2} = 5/54 \approx 0.093$

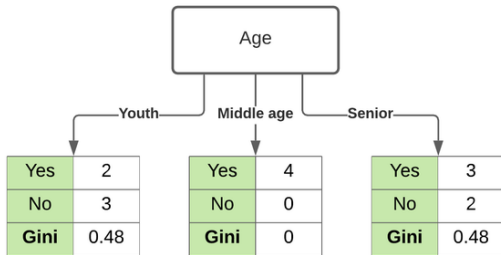
Cálculo de Gini:

$$G_i = 1 - (0^2 + 0.907^2 + 0.093^2)$$

$$G_i = 1 - (0 + 0.823 + 0.009) = 1 - 0.832 = 0.168$$

Interpretación

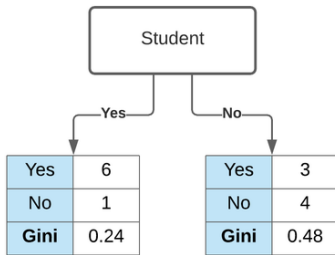
$G = 0.168$ indica impureza moderada. El nodo está dominado por versicolor (90.7%) pero tiene algunas virginicas (9.3%).



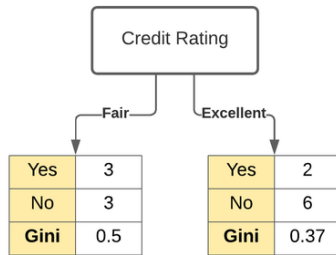
Gini Impurity for Age is 0.343



Gini Impurity for Income is 0.440



Gini Impurity for Student is 0.367



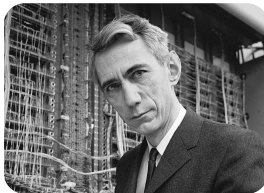
Gini Impurity for Credit Rating is 0.429

Best

Entropy: Definición desde la teoría de la información

Entropía: mide el grado de *incertidumbre* o *desorden* en un conjunto de datos.

- Concepto introducido en 1948 por *Claude Shannon* en su artículo *A Mathematical Theory of Communication*.
- La entropía cuantifica la cantidad *promedio de información necesaria* para describir una variable aleatoria.
- Cuanto mayor es la entropía, mayor es la incertidumbre; una buena división reduce esa entropía, creando nodos más puros.



“El problema fundamental de la comunicación es reproducir en un punto, exactamente o aproximadamente, un mensaje seleccionado en otro punto.”

— Claude E. Shannon

Entropy: Definición desde teoría de información

Fórmula:

$$H_i = - \sum_{k=1}^n p_{i,k} \log_2(p_{i,k})$$

Donde:

- $p_{i,k}$: proporción de clase k en nodo i
- $\log_2$: logaritmo base 2 (medida en bits)
- Convención: $0 \log_2(0) = 0$
- Rango: $[0, \log_2(n)]$
- $H_i = 0$ cuando hay una sola clase

Entropy: Definición desde teoría de información

Interpretación de Shannon

Número promedio de bits necesarios para codificar la información

Entropy: Ejemplo numérico y comparación

Mismo escenario anterior: 54 muestras (0 setosa, 49 versicolor, 5 virginica)

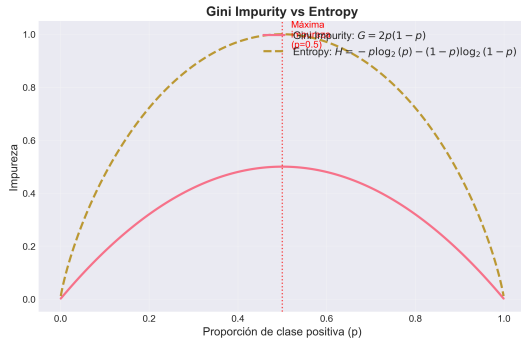
$$H_i = -(0 \cdot \log_2(0) + 0.907 \cdot \log_2(0.907) + 0.093 \cdot \log_2(0.093))$$

$$H_i = -(0 + 0.907 \cdot (-0.141) + 0.093 \cdot (-3.426))$$

$$H_i = -(-0.128 - 0.319) = 0.447 \text{ bits}$$

Comparación Gini vs Entropy:

- Gini: $G = 0.168$
- Entropy: $H = 0.447$



Information Gain: Reducción de entropía

El **Information Gain** mide cuánta incertidumbre se reduce al hacer un split.

Fórmula:

$$IG = H_{\text{parent}} - \sum_{j \in \{\text{left}, \text{right}\}} \frac{n_j}{n} H_j$$

Donde:

- H_{parent} : entropía del nodo padre
- H_j : entropía del hijo j
- n_j : número de muestras en hijo j
- n : número total de muestras

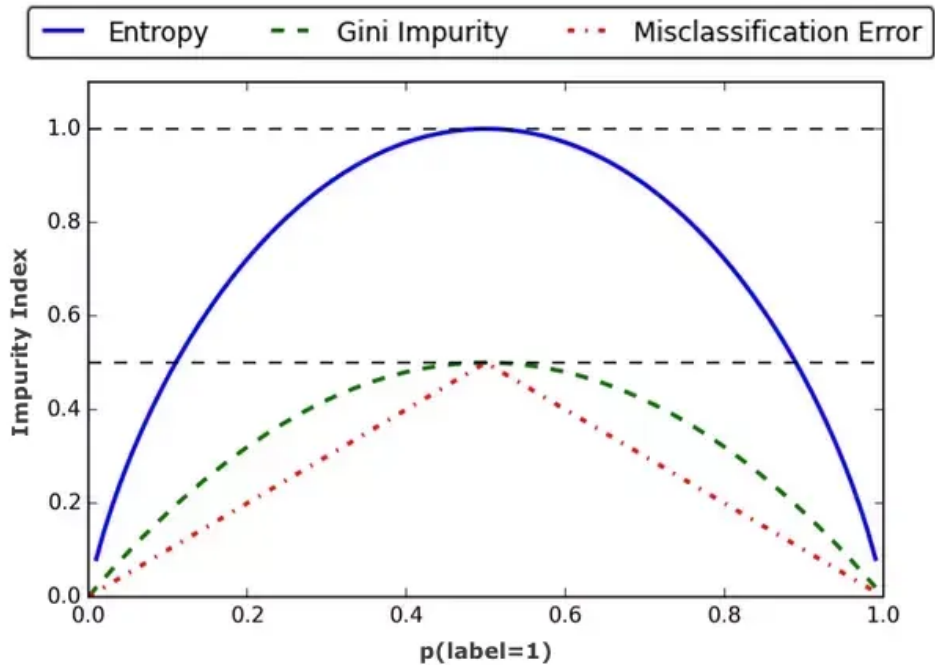
Information Gain: Reducción de entropía

Objetivo

Maximizar Information Gain = encontrar el split que más reduce la entropía

Equivalencia

Maximizar IG es equivalente a minimizar la entropía ponderada de los hijos



Gini vs Entropy: ¿Cuál usar?

Gini Impurity:

- Más rápido computacionalmente
- No requiere logaritmos
- Tiende a aislar clase más frecuente
- Default en *scikit-learn*
- Rango: $[0, 0.5]$ (binario)

Entropy:

- Computacionalmente más costoso
- Base teórica más sólida
- Tiende a árboles más balanceados
- Penaliza más fuertemente impureza
- Rango: $[0, 1]$ (binario)

Recomendación práctica

En la mayoría de casos producen árboles similares. Usar Gini por defecto (velocidad). Probar Entropy si se buscan árboles más balanceados.

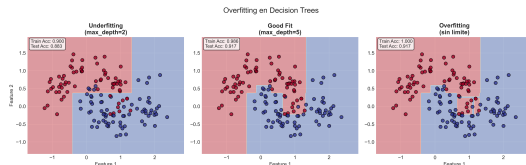
Regularización en Decision Trees: Controlando la Complejidad

El problema del overfitting

Sin restricciones, un árbol de decisión puede crecer hasta memorizar perfectamente el conjunto de entrenamiento.

Síntomas de overfitting:

- Árbol muy profundo
- Hojas con pocas muestras
- Error training ≈ 0
- Error test $\gg$ Error training
- Decisiones *boundary* muy irregulares



Modelos no paramétricos

Decision trees son modelos no paramétricos: el número de parámetros crece con los datos

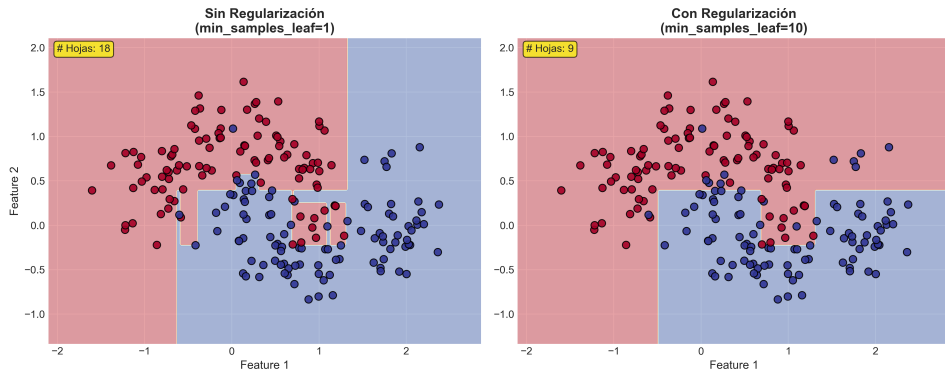
Hiperparámetros de regularización

Pre-pruning: Detener crecimiento durante construcción

- **max_depth:** Profundidad máxima del árbol
 - Típico: 3–10 para evitar overfitting
 - ∞ permite memorización completa
- **min_samples_split:** Mínimo de muestras para dividir nodo
 - Típico: 2–20
 - Valores altos $\rightarrow$ árboles más simples
- **min_samples_leaf:** Mínimo de muestras en hoja
 - Asegura generalización
 - Evita hojas con muestras individuales
- **max_leaf_nodes:** Número máximo de hojas
 - Control directo de complejidad
 - Alternativa a max_depth

Efecto de la regularización

Efecto de Regularización (`min_samples_leaf`)



Izquierda: Sin regularización (*overfitting*)

Derecha: Con `min_samples_leaf=4` (mejor generalización)

Ensemble Learning: La Sabiduría de las Multitudes

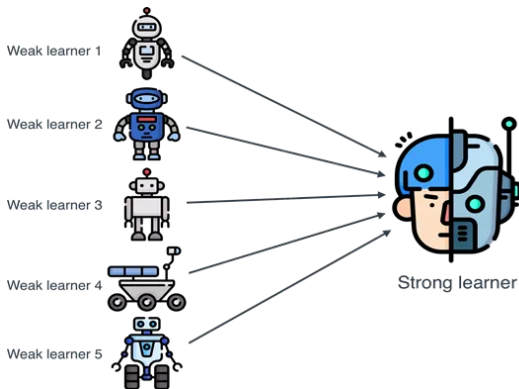
¿Qué es Ensemble Learning?

Principio fundamental: Combinar múltiples modelos (*weak learners*) para crear un predictor más robusto y preciso.

Analogía:

- Pregunta compleja a 1000 personas
- Promedia sus respuestas
- Resultado mejor que respuesta de experto

Wisdom of the Crowd



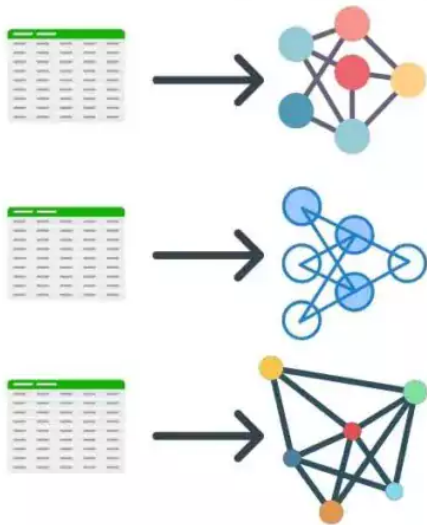
¿Qué es Ensemble Learning?

Fórmula general:

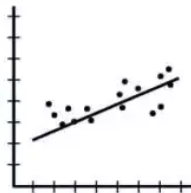
$$\hat{y}_{\text{ensemble}} = f(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M)$$

Donde f es agregación (promedio, votación, ponderado)

Different Models



Meta-Learner



Prediction

¿Por qué funciona Ensemble Learning?

Requisito clave: Los modelos deben ser DIVERSOS (errores no correlacionados)

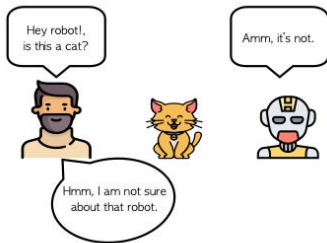
Matemática básica:

Para M modelos independientes con error ϵ cada uno:

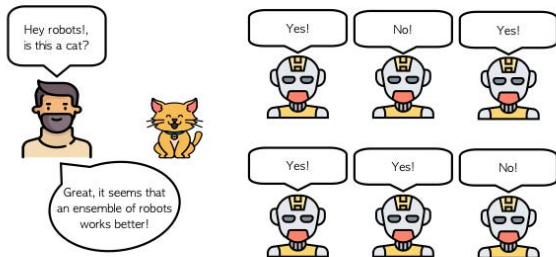
$$\text{Error}_{\text{ensemble}} \approx \frac{\epsilon}{\sqrt{M}}$$

Ley de los grandes números

Con suficientes *weak learners* independientes, el *ensemble* converge a la predicción correcta

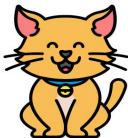


a) Traditional Learning



b) Ensemble Learning

Hey robots!,
is this a cat?



Great, it seems that
an ensemble of robots
works better!

Yes!



No!



Yes!



Yes!

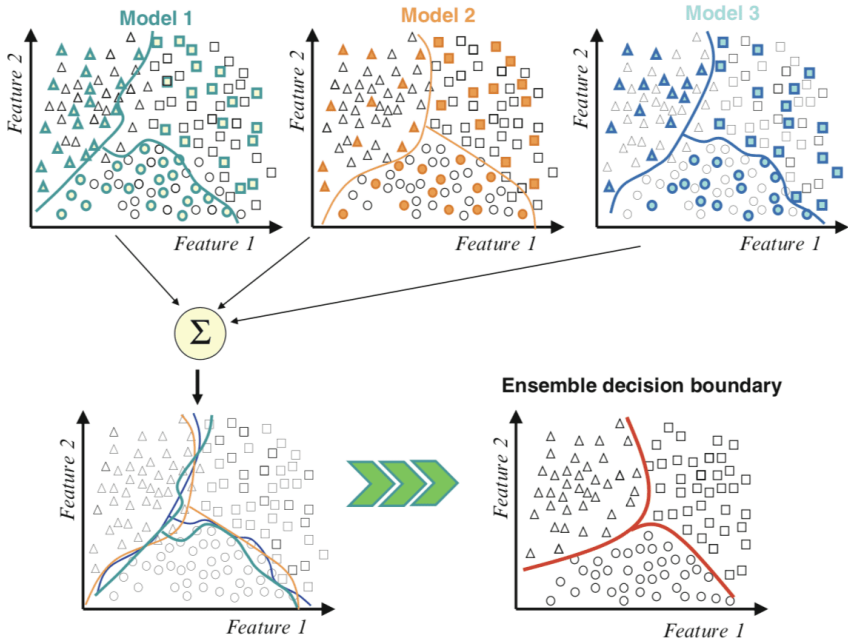


Yes!



No!





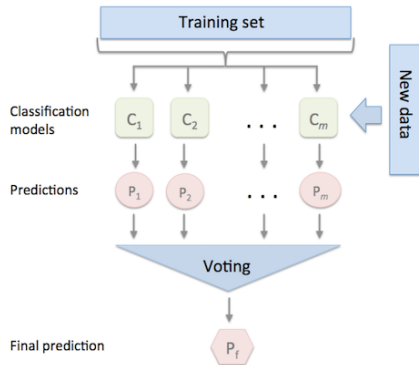
Votación: El Ensemble Más Simple

Voting Classifiers: Concepto

Combina predicciones de múltiples modelos diferentes mediante votación.

Idea central

Entrenar varios clasificadores distintos (SVM, Logistic Regression, Decision Tree, etc.) y agregar sus predicciones.



Voting Classifiers: Concepto

Hard Voting (clasificación):

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M)$$

Soft Voting (probabilidades):

$$\hat{y} = \underset{k}{\operatorname{argmax}} \sum_{m=1}^M p_m(y = k|x)$$

Regresión (promedio):

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m$$

Hard vs Soft Voting

Hard Voting:

- Cada modelo vota por una clase
- Gana la clase con más votos
- Simple pero puede perder información

Ejemplo:

- SVM: clase 1
- LR: clase 1
- DT: clase 0
- RF: clase 1

Resultado: clase 1 (3 votos vs 1)

Soft Voting:

- Promedia probabilidades
- Gana clase con mayor probabilidad promedio
- Usa confianza de cada modelo

Ejemplo:

- SVM: $P(1)=0.6$
- LR: $P(1)=0.55$
- DT: $P(1)=0.45$
- RF: $P(1)=0.7$

Resultado: $P(1)=0.575 \rightarrow$ clase 1

Requisito para Soft Voting

Todos los clasificadores deben tener método `predict_proba()`

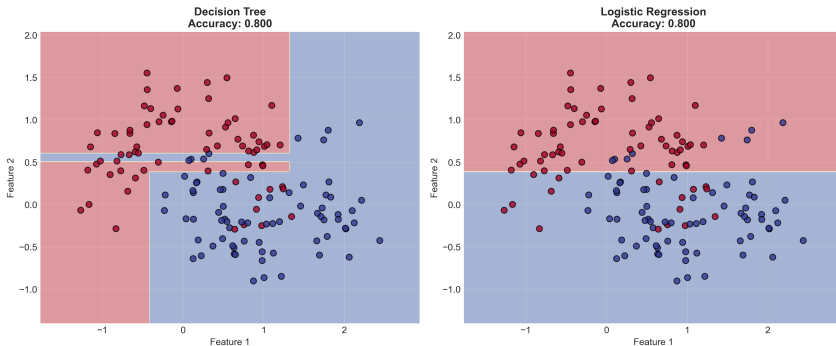
Requisito crítico: Diversidad

Problema: Si los clasificadores cometen los mismos errores, voting no ayuda.

Estrategias para diversidad:

- Usar algoritmos muy diferentes (SVM, trees, logistic regression, etc.)
- Entrenar en diferentes subconjuntos de features
- Usar diferentes hiperparámetros
- Entrenar en diferentes subconjuntos de datos

Voting Classifier: Diversidad de Modelos



Bagging: Bootstrap Aggregating

Bagging: Bootstrap Aggregating

Definición: Bagging combina dos ideas fundamentales:

- **Bootstrap:** crear múltiples datasets mediante muestreo con reemplazo.
- **Aggregating:** combinar las predicciones de varios modelos para reducir varianza.

Bagging: Bootstrap Aggregating

Algoritmo:

- 1 Para $m = 1, \dots, M$ (número de modelos):
 - Generar muestra bootstrap: $\mathcal{D}_m = \{(x^{(i)}, y^{(i)})\}$ (con reemplazo).
 - Entrenar modelo base h_m en $\mathcal{D}_m$.

2 Agregación final:

$$\hat{y}_{\text{bagging}} = \begin{cases} \frac{1}{M} \sum_{m=1}^M h_m(x) & \text{(regresión)} \\ \text{mode}(h_1(x), \dots, h_M(x)) & \text{(clasificación)} \end{cases}$$

Beneficio

Bagging reduce la **varianza**, produce modelos más **estables** y mejora la **generalización**.

Bagging

Original data



Bootstrap sample 1



Bootstrap sample 2



...

Bootstrap sample n



Bootstrapping

(Sampling with replacement)

Model 1

Model 2

...

Model n

Model Training

(Different models such as Decision Trees, SVM, Logistic can be used)

© AIML.com Research

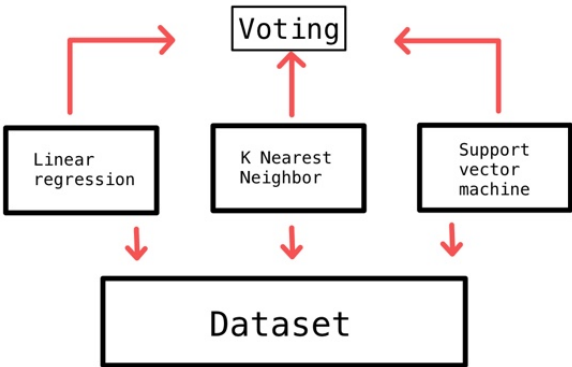
Ensemble Model

Aggregation

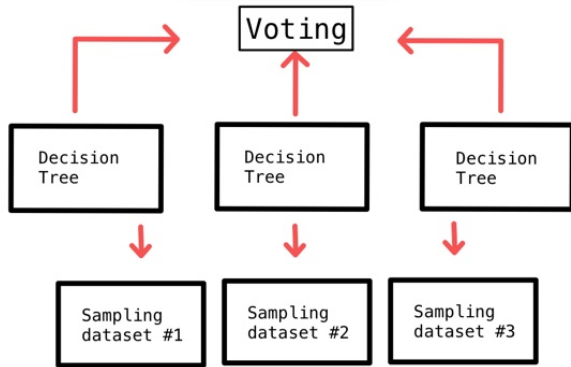
(Classification: Majority vote is taken
Regression: Average of outputs)

Bagging

Voting



Bagging



Bootstrap Sampling

Bootstrap: técnica de remuestreo **con reemplazo** a partir del dataset original.

Dataset original:

$$\mathcal{D} = \{x_1, x_2, \dots, x_n\}$$

Muestra bootstrap:

- Se seleccionan n elementos con reemplazo.
- Algunas observaciones se repiten.
- Otras no aparecen (muestras OOB).

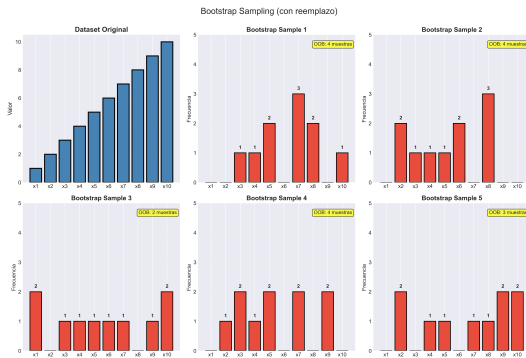


Figura 1: Ejemplo de remuestreo Bootstrap

Bootstrap Sampling

Probabilidad de selección:

$$P(x_i \text{ seleccionado}) = 1 - \left(1 - \frac{1}{n}\right)^n \approx 0.632$$

Out-of-Bag (OOB)

Aproximadamente el **37 % de los datos** no aparecen en cada muestra bootstrap y pueden usarse para **validación**.

Bosques Aleatorios

Random Forest: Bagging + Aleatoriedad

Definición: Ensemble de Decision Trees entrenados con *Bagging + random feature selection*.

Diferencias clave con *Bagging* básico:

- 1 **Base learner:** Siempre Decision Trees
- 2 **Feature subsampling:** En cada split, seleccionar subset aleatorio de features
- 3 **Decorrelación:** Reduce correlación entre árboles

Random Forest: Bagging + Aleatoriedad

Algoritmo:

- Para cada árbol m :
 - 1 Bootstrap sample $\mathcal{D}_m$
 - 2 Para cada nodo:
 - Seleccionar k features aleatorios de n totales
 - Encontrar mejor split entre esos k features
 - 3 Crear árbol sin poda ($\text{max_depth} = \text{None}$)

Feature Subsampling: Clave del éxito

¿Por qué es importante?

Sin feature subsampling (Bagging puro):

- Todos los árboles tienden a usar los mejores features en la raíz
- Árboles muy correlacionados
- Reducción de varianza limitada

Con feature subsampling (Random Forest):

- Fuerza uso de features subóptimas
- Árboles más diversos
- Mejor reducción de varianza

Número típico de features:

$$k = \begin{cases} \sqrt{n} & \text{(clasificación, default)} \\ n/3 & \text{(regresión)} \\ \log_2(n) & \text{(alternativa)} \end{cases}$$

Feature Importance: Interpretación y cuidados

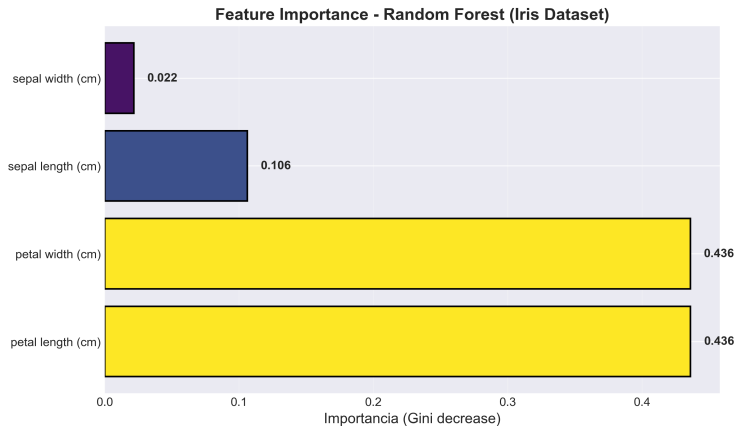
Random Forest proporciona medida de importancia de cada feature. **Ventajas:**

- Fácil de calcular (disponible automáticamente)
- Útil para selección de features
- Ayuda a interpretabilidad del modelo

Limitaciones:

- Sesgado hacia features de alta cardinalidad
- Sesgado hacia features numéricas
- No captura interacciones complejas
- Puede ser engañoso con features correlacionadas

Feature Importance: Interpretación y cuidados



Alternativa

Permutation Importance: más robusta pero más costosa computacionalmente

Hiperparámetros de Random Forest

Específicos de Random Forest:

- `n_estimators`: Número de árboles (típico: 100-500)
- `max_features`: Features por split (típico: $\sqrt{n}$)

Heredados de Decision Trees:

- `max_depth`: Profundidad máxima (None para árboles completos)
- `min_samples_split`: Mínimo para dividir (default: 2)
- `min_samples_leaf`: Mínimo en hoja (default: 1)
- `max_leaf_nodes`: Número máximo de hojas

De Bagging:

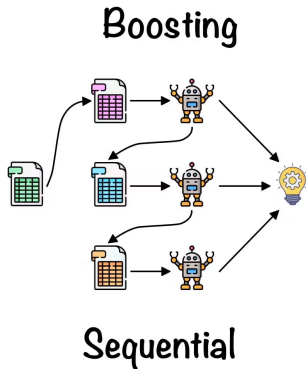
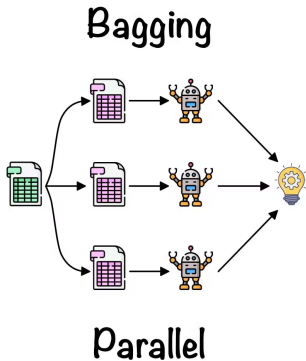
- `bootstrap`: Usar bootstrap sampling (default: True)
- `oob_score`: Calcular OOB score (default: False)
- `n_jobs`: Número de cores para paralelización (-1 = todos)

Boosting: Aprendizaje Secuencial

Boosting: Filosofía

Diferencia fundamental con Bagging:

- **Bagging:** modelos independientes en paralelo
- **Boosting:** modelos secuenciales, cada uno corrige errores del anterior



Boosting: Filosofía

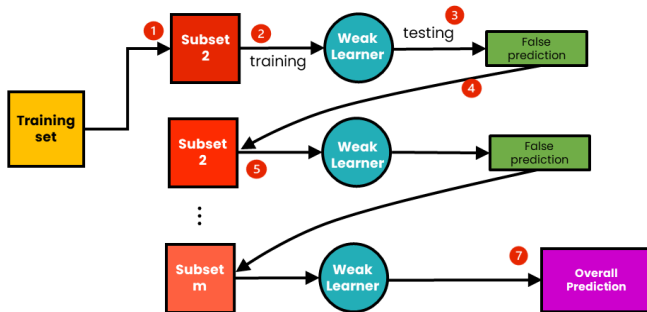
Principio:

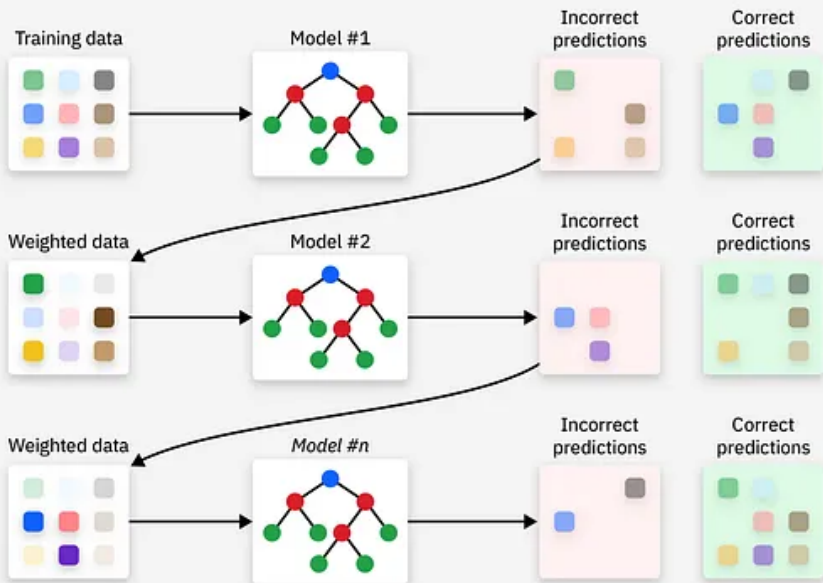
$$F_m(x) = F_{m-1}(x) + h_m(x)$$

Donde:

- F_m : modelo acumulado hasta iteración m
- h_m : weak learner en iteración m (enfocado en errores)

The Process of Boosting





Gradient Boosting: Optimización por Gradiente Descendente

Gradient Boosting: Concepto fundamental

Idea revolucionaria: *Boosting* como optimización en espacio de funciones.

Framework general:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

Donde:

- F_m : modelo acumulado
- h_m : nuevo *weak learner*
- ν : learning rate (shrinkage)

Objetivo: Minimizar función de pérdida

$$L(y, F(x)) = \sum_{i=1}^n \ell(y_i, F(x_i))$$

Insight clave

Cada h_m es un paso de *gradient descent* en el espacio de funciones

Gradient Boosting: Derivación matemática

Queremos minimizar:

$$L(F) = \sum_{i=1}^n \ell(y_i, F(x_i))$$

Expansión de Taylor de primer orden:

$$\ell(y_i, F_{m-1}(x_i) + h_m(x_i)) \approx \ell(y_i, F_{m-1}(x_i)) + h_m(x_i) \frac{\partial \ell(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$$

Definir pseudo-residuos:

$$r_{im} = - \frac{\partial \ell(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$$

Entonces:

$$L(F_m) \approx L(F_{m-1}) - \sum_{i=1}^n h_m(x_i) \cdot r_{im}$$

Para minimizar L , ajustar h_m a los pseudo-residuos r_{im}

Gradient Boosting: Algoritmo general

Inicialización:

$$F_0(x) = \operatorname{argmín}_{\gamma} \sum_{i=1}^n \ell(y_i, \gamma)$$

Para $m = 1$ a M :

1 Calcular pseudo-residuos:

$$r_{im} = - \left[\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

2 Ajustar modelo h_m a los residuos:

$$h_m = \operatorname{argmín}_h \sum_{i=1}^n (r_{im} - h(x_i))^2$$

3 Actualizar modelo: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$

Predicción final: $F_M(x)$

XGBoost: Extreme Gradient Boosting

XGBoost: idea y aportes

- **Modelo aditivo de árboles:** suma de árboles poco profundos que corrigen el error anterior.
- **Segundo orden:** usa gradientes (g_i) y Hessianos (h_i) del loss para decidir splits y valores de hoja.
- **Regularización explícita:** penaliza hojas y pesos para controlar complejidad.
- **Sparsity-aware y missing handling:** aprende la rama por defecto para valores faltantes.
- **Eficiencia:** paraleliza la búsqueda de splits y escala a datos grandes (out-of-core, caché).

XGBoost

Función objetivo (con regularización)

$$\mathcal{L}(\phi) = \sum_{i=1}^n \ell(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2$$

Significado:

- ℓ : pérdida (p.ej., MSE, log loss).
- f_k : árbol k con T hojas y valores w_j .
- γ : costo por añadir hojas ($\rightarrow$ poda natural).
- λ : regularización L2 sobre los valores de hoja ($\rightarrow$ evita sobreajuste).

Aprendizaje por segundo orden

En la iteración t se añade f_t sobre la predicción previa:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Aproximación de Taylor (2º orden):

$$\ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx \ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)$$

Derivadas clave:

$$g_i = \left. \frac{\partial \ell}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}, \quad h_i = \left. \frac{\partial^2 \ell}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}$$

Intuición: el árbol nuevo f_t se ajusta a una versión “cuadrática” del error actual, usando g_i y h_i .

Fórmulas por nodo: pesos y *gain*

Agrupando por hoja j (con I_j las instancias en esa hoja):

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Peso óptimo de hoja:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Criterio de división (*gain*) para un split en dos hijos L, R :

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma$$

Lectura: mayor Gain $\Rightarrow$ mejor split; γ y λ frenan divisiones débiles y valores extremos.

XGBoost: Hiperparámetros principales

Control del árbol:

- `max_depth`: profundidad máxima (3-10)
- `min_child_weight`: suma mínima de Hessian en hoja
- `gamma`: reducción mínima de loss para split

Boosting:

- `n_estimators`: número de árboles (100-1000)
- `learning_rate`: shrinkage (0.01-0.3)
- `subsample`: fracción de muestras (0.5-1.0)
- `colsample_bytree`: fracción de features (0.5-1.0)

Regularización:

- `reg_alpha`: L1 regularization (α)
- `reg_lambda`: L2 regularization (λ)

LightGBM: Light Gradient Boosting Machine

LightGBM: Optimización para eficiencia

LightGBM por Microsoft (2017)

Objetivo: Más rápido y eficiente que XGBoost

Innovaciones principales:

- 1 Gradient-based One-Side Sampling (GOSS)
- 2 Exclusive Feature Bundling (EFB)
- 3 Histogram-based algorithm
- 4 Leaf-wise tree growth



Ventaja clave

Mantiene accuracy mientras reduce drásticamente tiempo de entrenamiento

LightGBM: ideas clave (GOSS + EFB)

Objetivo: Igualar/ganar en accuracy a XGBoost con mucho menos tiempo/memoria.

GOSS (Gradient-based One-Side Sampling)

- Mantener top $a\%$ con $|g_i|$ grandes; muestrear $b\%$ del resto.
- Reponderar los muestreados por $\frac{1-a}{b}$ para conservar la distribución del gradiente.
- Usa $(a+b)\%$ de datos por split $\Rightarrow$ gran aceleración sin sesgar el criterio.

EFB (Exclusive Feature Bundling)

- Muchas features son *sparse* y mutuamente exclusivas (p.ej., one-hot).
- Construye grafo de conflictos y agrupa en *bundles*; usa *offsets* para separar.
- Reduce dimensionalidad: $m \rightarrow k$ con $k \ll m$ (misma información, menos costo).

LightGBM: Histogramas & crecimiento leaf-wise

Histogram-based

- Discretiza cada feature en k bins (típico ≈ 255).
- Construye histogramas de $\sum g, \sum h$ por bin y busca splits sólo en bordes de bins.
- Ventajas: memoria $O(k)$, menos comparaciones (k vs. valores únicos), acceso cache-friendly.

Leaf-wise tree growth

- Elige siempre la *hoja* con mayor *gain* para expandir (no nivel por nivel).
- Suele lograr mayor reducción de pérdida con menos splits (más profundo donde importa).
- **Trade-off**: mayor riesgo de sobreajuste $\Rightarrow$ controlar con `max_depth` y `num_leaves`.

Resumen

GOSS + EFB + histogramas + leaf-wise $\Rightarrow$ **entrenamiento mucho más rápido** con **accuracy competitivo**.

LightGBM: Hiperparámetros clave

Específicos de LightGBM:

- `num_leaves`: Número máximo de hojas (31 default)
- `max_bin`: Número de bins para histogramas (255 default)
- `min_data_in_leaf`: Mínimo de muestras en hoja

Similares a XGBoost:

- `learning_rate`, `n_estimators`
- `max_depth`: -1 para sin límite
- `subsample`, `colsample_bytree`
- `reg_alpha`, `reg_lambda`

Recomendación inicial

$\text{num_leaves} \leq 2^{\text{max_depth}}$ para prevenir overfitting

Categorical Boosting

CatBoost: Manejo nativo de categóricas

CatBoost por Yandex (2017)

Problema que resuelve: Features categóricas de alta cardinalidad

Innovaciones:

- 1 **Ordered Target Statistics** (manejo de categóricas)
- 2 **Symmetric Trees** (oblivious trees)
- 3 **Ordered boosting** (previene target leakage)
- 4 Excelentes defaults (menos tuning)



CatBoost

Uso ideal

Datasets con muchas features categóricas, especialmente alta cardinalidad

Target Leakage y solución: Ordered Target Statistics

Problema (Target Encoding ingenuo):

$$\hat{x}_k(c) = \frac{\sum_{j: x_j^k=c} y_j}{\sum_{j: x_j^k=c} 1} \Rightarrow \text{usa su propio } y_i \text{ y fuga información}$$

Solución (CatBoost, Ordered Target Statistics): para la muestra i en una permutación σ ,

$$\hat{x}_k^i = \frac{\sum_{\substack{j: \sigma(j) < \sigma(i) \\ x_j^k = x_i^k}} y_j + aP}{\sum_{\substack{j: \sigma(j) < \sigma(i) \\ x_j^k = x_i^k}} 1 + a}$$

- Solo usa *muestras anteriores* en la permutación ($\Rightarrow$ sin mirar y_i).
- P : prior (promedio global de y), a : peso del prior (p.ej., $a = 1$).

Mini-ejemplo (color=red):

- En la posición 1: no hay previas $\Rightarrow \hat{x} = P$.
- En la posición 3: usa solo las previas con red (no su propio y).

CatBoost: Ordered Boosting y Árboles Simétricos

Ordered Boosting (evita fuga en los gradientes):

- Para cada árbol m , genera una permutación σ_m .
- Calcula residuos con el modelo previo, usando solo información *anterior* en σ_m .
- Entrena m sobre esos residuos ordenados. $\Rightarrow$ gradientes “causales”.

Árboles Simétricos (Oblivious Trees):

- Todos los nodos del mismo nivel comparten el mismo split (feature/umbral).
- Estructura balanceada: profundidad $d \Rightarrow 2^d$ hojas.
- *Pros*: predicción muy rápida ($O(d)$), menor sobreajuste, robusto para producción.
- *Contras*: menos flexibles que árboles asimétricos.

Resumen

Ordered stats/boosting $\Rightarrow$ elimina fugas; árboles simétricos $\Rightarrow$ velocidad y estabilidad.

CatBoost: Hiperparámetros

Específicos de CatBoost:

- `iterations`: Número de árboles (equivale a `n_estimators`)
- `depth`: Profundidad de árboles (típico: 6-10)
- `cat_features`: Índices de features categóricas
- `one_hot_max_size`: Threshold para one-hot vs target encoding

Estándar:

- `learning_rate`, `subsample`
- `reg_lambda`: L2 regularization
- `random_strength`: Aleatoriedad en split selection

Ventaja CatBoost

Defaults muy buenos, requiere menos tuning que XGBoost/LightGBM

XGBoost vs LightGBM vs CatBoost

Comparación: Características

Característica	XGBoost	LightGBM	CatBoost
Tree Growth	Level-wise	Leaf-wise	Level-wise
Cat. Features	Manual	Manual	Native
Velocidad	Media	Rápida	Media
Memoria	Alta	Baja	Media
Overfitting	Medio	Alto	Bajo
GPU Support	Sí	Sí	Sí
Default Params	Buenos	Necesita tuning	Excelentes
Missing Values	Sí	Sí	Sí

Comparación: Cuándo usar cada uno

XGBoost:

- Balance general de accuracy y velocidad
- Datasets medianos ($< 100k$ muestras)
- Cuando interpretabilidad del modelo es importante
- Comunidad grande, documentación extensa

LightGBM:

- Datasets grandes ($> 100k$ muestras)
- Prioridad en velocidad de entrenamiento
- Muchas features (> 100)
- Memoria limitada

CatBoost:

- Muchas features categóricas
- Alta cardinalidad en categóricas
- Menos tiempo para hyperparameter tuning
- Necesidad de robustez contra overfitting

Conclusiones

Best Practices

Workflow recomendado:

- 1 **Baseline:** Comenzar con Random Forest
- 2 **Boosting:** Probar XGBoost/LightGBM/CatBoost
- 3 **Feature Engineering:** Crucial para GBDT
- 4 **Hyperparameter Tuning:**
 - Usar Bayesian Optimization (Optuna, Hyperopt)
 - Cross-validation robusto
- 5 **Early Stopping:** Siempre activado
- 6 **Ensemble:** Combinar múltiples modelos

Resumen: Decision Trees & Ensemble Learning

Conceptos clave:

- Decision Trees: Interpretables pero inestables
- Bagging: Reduce varianza mediante agregación
- Random Forest: Bagging + feature subsampling
- Boosting: Reduce sesgo mediante aprendizaje secuencial
- XGBoost: Second-order optimization + regularización
- LightGBM: Eficiencia mediante GOSS + EFB + histograms
- CatBoost: Categóricas nativas + ordered statistics

Mensaje central

Ensemble methods convierten weak learners en strong learners mediante agregación inteligente

¡Muchas gracias por su atención!

¿Preguntas?



Contacto: Marco Teran
webpage: marcoteran.github.io/
e-mail: mtteranl@eafit.edu.co